Preprints.org

Article

Not peer-reviewed version

Multi-Objective Scheduling for Large Language Model Inference with Prompt-Level Cost Prediction and SLO Awareness

Jiajing Liao , Feng Chang , Yihan Xue , Tianjian Xia , Zeyu Huang , Yuxiao Wang *

Posted Date: 20 April 2026

doi: 10.20944/preprints202604.1399.v1

Keywords: LLM inference; carbon-aware scheduling; multi-tenant cloud; prompt complexity; SLO

Preprints.org is a free multidisciplinary platform providing preprint service that is dedicated to making early versions of research outputs permanently available and citable. Preprints posted at Preprints.org appear in Web of Science, Crossref, Google Scholar, Scilit, Europe PMC.

Copyright: This open access article is published under a Creative Commons CC BY 4.0 license, which permit the free download, distribution, and reuse, provided that the author and preprint are cited in any reuse.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

Disclaimer/Publisher's Note: The statements, opinions, and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions, or products referred to in the content.

Article

Multi-Objective Scheduling for Large Language Model Inference with Prompt-Level Cost Prediction and SLO Awareness

Jiajing Liao [1], Feng Chang [2], Yihan Xue [3], Tianjian Xia [4], Zeyu Huang [5] and Yuxiao Wang [6,*]

[1] University of Florida, Gainesville, USA
[2] North Dakota State University, Fargo, USA
[3] University of Southern California, Los Angeles, USA
[4] Syracuse University, Syracuse, USA
[5] University of California, Irvine, Irvine, USA
[6] University of Pennsylvania, Philadelphia, USA
* Correspondence: yx99.wang@gmail.com

Abstract

Large language model (LLM) inference in multi-tenant clouds is becoming an increasingly important contributor to data-center carbon emissions, yet existing carbon-aware scheduling techniques target long-running training jobs and are ill-suited for the short, bursty, SLO-sensitive nature of online serving. We propose CAPS (Carbon–Aware Prompt Scheduling), an online bi-objective scheduler that jointly optimizes goodput and per-request carbon cost for multi-tenant LLM inference. CAPS first employs a lightweight prompt complexity predictor to estimate token generation cost and latency risk for each incoming request. It then combines real-time grid carbon intensity, GPU energy profiles, and per-tenant SLO tiers to route each request to one of three execution pools: a low-latency pool, a low-carbon pool, or a delay-tolerant batch pool. A composite reward function balances goodput, carbon emissions, and SLO violation rate. In trace-driven simulations using public conversation traces and regional carbon intensity data, CAPS reduces average carbon emissions per 1K generated tokens by 26.8% compared to round-robin scheduling while achieving an SLO attainment rate that matches or exceeds a dedicated SLO-aware baseline.

Keywords: LLM inference; carbon-aware scheduling; multi-tenant cloud; prompt complexity; SLO

Introduction

The rapid adoption of large language models (LLMs) such as GPT-4, LLaMA, and Claude has shifted the computational bottleneck from one-time training to continuous inference serving [1]. Recent studies suggest that, in large-scale deployments, the cumulative operational cost of LLM inference can rival or exceed training costs over time [2]. Because inference workloads run around the clock, their cumulative carbon footprint is substantial and closely tied to the real-time carbon intensity of the electricity grid [3].

Carbon-aware scheduling has proven effective in reducing the environmental impact of ML workloads. Frameworks such as CarbonScaler [4] and GREEN [5] exploit temporal and spatial flexibility to shift batch training jobs toward periods or regions with cleaner electricity. GreenCourier [6] extends Kubernetes scheduling with marginal emission scores for serverless functions. Carbon Explorer [7] provides a holistic design space for carbon-aware datacenters. However, these approaches primarily target long-running, delay-tolerant workloads. Training jobs offer minutes-to-

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

hours of temporal flexibility, whereas online LLM inference requests are constrained by tight TTFT and tail-latency SLOs, making traditional workload shifting inapplicable at the request level.

On the serving side, recent systems have made significant advances in LLM inference efficiency. Orca [8] introduced continuous batching at the iteration level, while vLLM [9] added PagedAttention for efficient GPU memory management. Sarathi-Serve [10] proposed chunked prefills to balance throughput and tail latency. Splitwise [11] demonstrated the benefits of separating the prefill and decode phases across heterogeneous hardware. SGLang [12] optimized structured generation programs. Yet none of these works incorporates carbon cost as an explicit optimization objective.

This paper bridges the gap between carbon-aware scheduling and LLM serving by proposing **CAPS**, a scheduler that is simultaneously *prompt-aware*, *carbon-aware*, and *tenant-aware*.

Our contributions are as follows:

A lightweight prompt complexity predictor that estimates per-request token generation cost and latency risk prior to execution.

A three-pool routing architecture that exploits heterogeneous SLO tolerance among tenants to shift flexible requests toward low-carbon periods.

A composite reward function that jointly optimizes goodput, average carbon emissions, and SLO violation rate.

A simulation-based evaluation on public conversation traces and grid carbon intensity data demonstrates meaningful carbon savings with modest SLO impact.

Methodology Foundations and Motivation

The methodological design of CAPS is grounded in the convergence of carbon-aware system optimization, adaptive workload characterization, and multi-objective decision-making frameworks. Unlike traditional carbon-aware approaches that rely on coarse-grained workload shifting, the proposed method operates at the request level, requiring fine-grained modeling of both computational cost and latency risk under strict service-level objectives (SLOs). This necessitates integrating forecasting, representation learning, and online scheduling strategies into a unified framework.

A key foundation of the proposed approach lies in carbon intensity modeling and prediction. Multi-day grid carbon forecasting techniques provide the necessary temporal signals for anticipating energy-related emissions and enabling proactive scheduling decisions [13]. These forecasting capabilities establish the basis for incorporating real-time and near-future carbon intensity into the scheduling loop. However, prior work primarily targets batch workloads with significant temporal flexibility, leaving a gap in handling latency-sensitive inference requests. CAPS extends this paradigm by embedding carbon awareness into per-request routing decisions, rather than relying on coarse job deferral strategies.

At the workload modeling level, accurately estimating the computational and latency characteristics of incoming requests is essential. Prior research in adaptive anomaly detection and meta-learning demonstrates how models can generalize across evolving data patterns and infer latent complexity under limited prior information [14,15]. Similarly, sequence-based and recurrent modeling approaches highlight the importance of capturing temporal dependencies and structural patterns in dynamic inputs [16]. These methodologies inform the design of the lightweight prompt complexity predictor in CAPS, which approximates token generation cost and latency risk before execution. Unlike traditional performance modeling techniques that rely on static profiling, this predictor adopts a learning-based abstraction capable of adapting to heterogeneous and evolving prompt distributions.

The representation of heterogeneous inputs and contextual signals further draws from advances in cross-modal and context-aware learning. Joint representation learning across multiple modalities has been shown to enhance predictive performance by integrating complementary information sources [17], while retrieval-augmented and context-aware frameworks improve robustness under evolving data conditions [18]. These insights motivate the integration of prompt features, system

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

state, and carbon signals into a unified decision space. Nevertheless, existing approaches typically focus on improving prediction accuracy rather than enabling real-time scheduling decisions under strict constraints, which is a limitation addressed by the CAPS routing mechanism.

Handling uncertainty and dynamic system behavior is another critical methodological component. Self-supervised learning and uncertainty-aware modeling techniques provide mechanisms for robust decision-making under noisy and incomplete observations [19,20]. In parallel, anomaly detection methods for time-series and system logs emphasize the importance of capturing distribution shifts and transient behaviors in large-scale systems [21]. These approaches collectively inform the robustness considerations in CAPS, particularly in mitigating the risk of SLO violations when routing requests to different execution pools. By implicitly modeling uncertainty in latency and workload characteristics, CAPS improves the reliability of its scheduling decisions.

The design of the scheduling objective is strongly influenced by advances in multi-objective optimization and decision-making. Recent work on calibrated multi-objective ranking demonstrates how competing objectives such as utility, fairness, and robustness can be jointly optimized through principled objective design [22]. Reinforcement learning-based resource management further highlights the effectiveness of learning policies that balance multiple system-level goals under dynamic conditions [23]. These methodologies directly inspire the composite reward function in CAPS, which simultaneously optimizes goodput, carbon emissions, and SLO adherence. Unlike prior approaches that treat these objectives independently, CAPS formulates them within a unified optimization framework, enabling coordinated trade-offs at runtime. In addition, system-level scheduling and resource allocation strategies contribute to the architectural design of CAPS. Shared representation learning and multi-task forecasting in cloud-native environments illustrate how resource contention and workload heterogeneity can be jointly modeled to improve system efficiency [24]. Drift-aware spatiotemporal modeling further emphasizes the need to adapt to evolving dependencies in large-scale distributed systems [25]. These insights underpin the three-pool routing architecture, where requests are dynamically assigned to execution pools based on their predicted characteristics and system conditions. Compared with monolithic scheduling strategies, this design enables differentiated handling of latency-critical and delay-tolerant workloads.

Finally, the incorporation of hierarchical and memory-driven decision processes provides additional support for modeling long-term system behavior and historical context. Memory-augmented frameworks demonstrate how past observations can inform future decisions in dynamic environments [26]. Although CAPS focuses on short-term scheduling, these ideas influence the use of historical workload patterns and system metrics in guiding routing policies. In summary, the methodology of CAPS is built upon a synthesis of carbon forecasting, adaptive workload modeling, multi-objective optimization, and distributed system scheduling. While prior work provides strong foundations in each of these areas, they largely operate in isolation or target different workload regimes. The key innovation of this work lies in integrating these methodologies into a unified, prompt-aware and carbon-aware scheduling framework tailored to the unique constraints of multi-tenant LLM inference, enabling fine-grained trade-offs between efficiency, sustainability, and service quality.

System Design

Figure 1 presents the overall architecture of CAPS, which comprises four main components: a prompt complexity predictor, a carbon and energy monitor, a tenant SLO registry, and a bi-objective scheduler that routes each request to one of three GPU execution pools.

Prompt Complexity Predictor

For each incoming request r_i with input token count n_i^{in}, we predict the expected output token count $\hat{n}_i^{\text{out}}$ and the associated latency risk $\hat{d}_i$ using a lightweight gradient-boosted regression model (LightGBM with 64 trees and max depth 6). The features include input length, conversation

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

turn count, estimated task type (e.g., classification, generation, or code), determined by keyword matching on the system prompt, and historical statistics for the tenant.

The predicted token generation cost is:

$$\hat{c}_i = \alpha_p \cdot n_i^{\text{in}} + \alpha_d \cdot \hat{n}_i^{\text{out}} \quad (1)$$

where α_p and α_d are hardware-specific coefficients for the prefill and decode phases, respectively. The latency risk $\hat{d}_i$ estimates the probability that request r_i will exceed its SLO deadline under the current system load.

We train the predictor on a separate prefix of the workload trace (the first three days) using an 80/10/10 train/validation/test split; the scheduling evaluation uses only the subsequent 7-day simulation window, ensuring no temporal overlap between predictor training and experimental evaluation. On the held-out test set, the output-length regression achieves a mean absolute error (MAE) of 38.6 tokens (MAPE 18.2%), and the SLO-risk binary classifier achieves an AUROC of 0.87. A single prediction takes less than 0.1 ms on CPU, which is negligible relative to the per-request GPU execution time of tens to hundreds of milliseconds.

To enable fine-grained resource allocation across heterogeneous workloads, CAPS builds upon the token-level GPU pooling mechanism of Zhuang et al. [27], which slices GPU resources across concurrent models. We adopt this mechanism to support flexible execution pools and leverage it to dynamically assign requests based on predicted complexity and carbon cost. CAPS further builds upon the token-level scheduling framework of Zeng et al.[28], which interleaves execution across models to improve utilization. We adopt this scheduling abstraction and extend it by incorporating carbon intensity and SLO constraints into routing decisions, enabling the scheduler to jointly optimize performance and environmental cost. To guide adaptive decision-making under uncertainty, CAPS incorporates the uncertainty-aware predictive modeling of Zhu et al.[29], which quantifies workload uncertainty for autoscaling. We adopt this principle to model uncertainty in prompt complexity and carbon signals, leveraging it within the bi-objective scheduler to balance goodput, carbon emissions, and SLO violations.

Carbon and Energy Monitor

The monitor maintains a rolling window of grid carbon intensity $\gamma(t)$ sourced from public APIs such as Electricity Maps and CarbonCast forecasts. GPU power draw $P_{\text{gpu}}(u)$ is modeled as a function of utilization u:

$$P_{\text{gpu}}(u) = P_{\text{idle}} + (P_{\text{peak}} - P_{\text{idle}}) \cdot u^{\eta} \quad (2)$$

where $\eta \approx 0.7$ captures the sub-linear power scaling observed in inference workloads [2]. The instantaneous carbon cost of executing request r_i is then:

$$\hat{E}_c(r_i, t) = \gamma(t) \cdot P_{\text{gpu}}(u_i) \cdot \tau_i \quad (3)$$

where τ_i is the estimated execution time derived from $\hat{c}_i$.

Bi-Objective Scheduler and Pool Routing

The scheduler maintains three GPU pools:

Low-Latency Pool (LP): High-performance GPUs dedicated to Gold-tier requests. Scheduling policy: first-come, first-served with preemptive priority.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

Low-Carbon Pool (CP): GPUs that preferentially execute during low-carbon windows. Silver-tier requests are admitted when $\gamma(t) \le \gamma_{\text{thresh}}$; otherwise they queue.

Delay-Tolerant Batch Pool (BP): Accumulates Bronze-tier requests and dispatches them in large batches during predicted low-carbon windows, maximizing throughput per unit carbon.

We define *goodput* G as the number of output tokens generated within SLO deadlines per second (tok/s). For each request r_i with SLO tier s_i, the scheduler evaluates a routing score for each candidate pool p:

$$\text{score}(r_i, p) = \alpha \cdot \hat{G}(r_i, p) - \beta \cdot \hat{E}_c(r_i, p) - \lambda \cdot \hat{V}(r_i, p) \qquad (4)$$

where $\hat{G}$ is the expected goodput contribution, $\hat{E}_c$ is the estimated carbon cost from Eq. (3), and $\hat{V}$ is the estimated SLO violation probability. Crucially, when evaluating the BP pool, the scheduler uses a *predicted future* carbon intensity $\hat{\gamma}(t_{\text{start}})$ from CarbonCast forecasts rather than the current $\gamma(t)$, since BP requests will execute at a later, potentially lower-carbon time slot. This lookahead is what enables the BP pool to achieve carbon savings; for LP and CP, the current $\gamma(t)$ is used because execution begins immediately or nearly so.

The weights α, β, λ are configurable; in our experiments, we set $\alpha = 1.0$, $\beta = 0.5$, $\lambda = 2.0$ to prioritize SLO compliance while still incentivizing carbon savings. The request is routed to the pool $p^* = \arg\max_p \text{score}(r_i, p)$.

Algorithm 1 summarizes the online scheduling loop.

Algorithm 1: CAPS Online Scheduling

Input: Request stream $\{r_i\}$, carbon signal $\gamma(t)$, forecast $\hat{\gamma}(\cdot)$, SLO registry

Output: Pool assignment for each r_i

```
for each arriving request r_i do
    (ĉ_i, d̂_i) ← PredictComplexity(r_i)
    s_i ← LookupSLO(r_i.tenant)
    γ_cur ← GetCarbonIntensity()
    γ̂_bp ← GetForecastedCI(t_low^next)
    for each pool p ∈ {LP, CP, BP} do
        Compute score(r_i, p) via Eq. (4)
        // Use γ̂_bp for BP; γ_cur for LP, CP
    end
    Assign r_i → arg max_p score(r_i, p)
end for
```

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

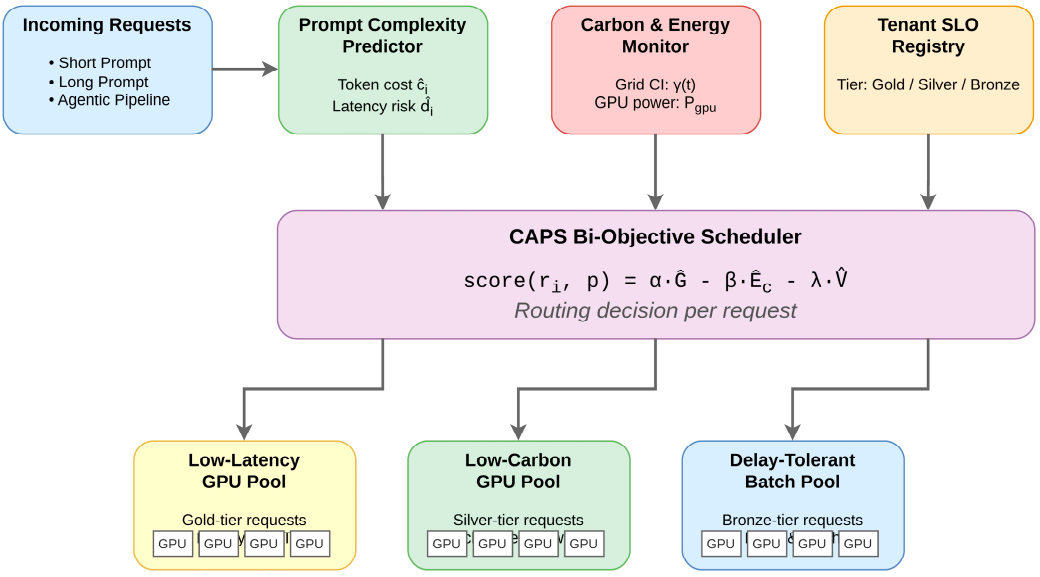


Figure 1. Overview of the CAPS system architecture.

Experimental Evaluation

Setup

We implement a discrete-event simulator that models a multi-tenant LLM inference cluster with 24 NVIDIA A100 GPUs (80 GB) organized into the three pools described in Section III (8 GPUs each by default). We set $P_{\text{idle}} = 100$ W and $P_{\text{peak}} = 400$ W per GPU, consistent with published A100 specifications [2].

Workload. We derive input-length and output-length distributions from the publicly available ShareGPT conversation corpus. From this corpus, we construct a synthetic workload comprising three request categories: *short prompts* ($n^{\text{in}} < 256$, 50% of traffic), *long prompts* ($256 \le n^{\text{in}} < 2048$, 35%), and *agentic pipelines* ($n^{\text{in}} \ge 2048$, 15%). Output lengths are sampled from a log-normal distribution fitted to the corpus ($\mu = 4.8$, $\sigma = 1.1$ in log-token space). Tenant SLO tiers are assigned as Gold (30%), Silver (40%), and Bronze (30%). Request arrivals follow a Poisson process at the specified rate.

Carbon intensity trace. We use hourly carbon intensity data from Electricity Maps for the CAISO region over August 5–11, 2024, exhibiting a diurnal range of approximately 90–520 gCO_2/kWh.

Baselines. We compare CAPS against: (1) **Round Robin (RR)**: requests are distributed cyclically across all 24 GPUs; (2) **Earliest Deadline First (EDF)**: requests are scheduled by SLO deadline; (3) **SLO-Aware**: a scheduler that routes requests to pools based solely on SLO tier without carbon awareness; and (4) **Carbon-Aware**: a scheduler that defers all requests to low-carbon windows without considering SLO heterogeneity. All baselines share the same pool topology and GPU count for fair comparison. We also tested nearby pool splits (6/10/8 and 10/6/8) and observed qualitatively similar trends; we report the balanced 8/8/8 configuration throughout.

Unless otherwise stated, all reported results are averaged over 5 independent simulator runs with different random seeds; standard deviations are below 1.5% for all metrics and are omitted for readability.

Main Results

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

Figure 2 presents SLO attainment, carbon emissions per 1K generated tokens, and P95 tail latency across increasing request rates. Table I reports summary metrics at the representative load of 10 req/s.

At 10 req/s, CAPS achieves an SLO attainment rate of 83.5%, which is 1.4 percentage points *higher* than the SLO-Aware baseline (83.5% vs. 82.1%), demonstrating that carbon-aware routing does not degrade service quality in our setting. The purely Carbon-Aware scheduler, which lacks SLO differentiation, drops to 63.2%. Meanwhile, CAPS reduces carbon per 1K tokens by 26.8% relative to Round Robin (10.6 vs. 14.5 ^{gCO_2}) and trails the Carbon-Aware baseline by only 2.9%. At the same load, CAPS reduces P95 latency by 37.3% relative to the Carbon-Aware baseline (640 vs. 1020 ms), confirming that prompt-aware routing effectively avoids penalizing latency-sensitive requests.

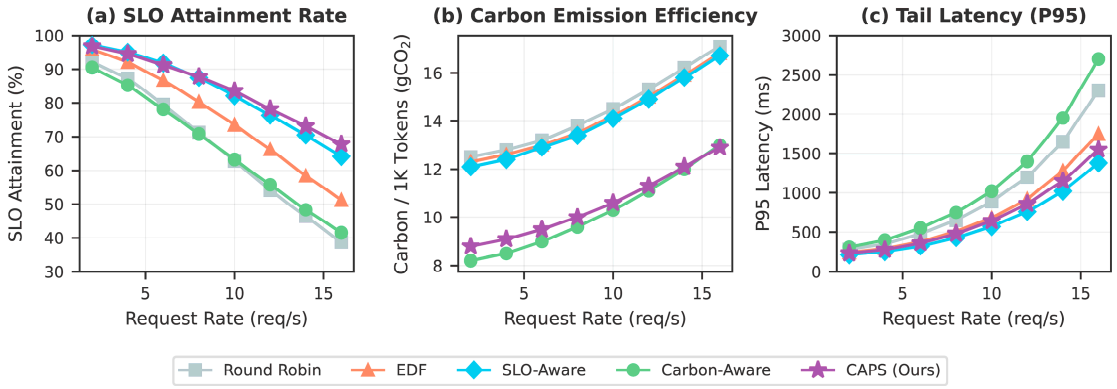


Figure 2. Performance comparison across request rates. CAPS achieves carbon efficiency close to the Carbon-Aware baseline while maintaining SLO attainment comparable to or exceeding the SLO-Aware baseline.

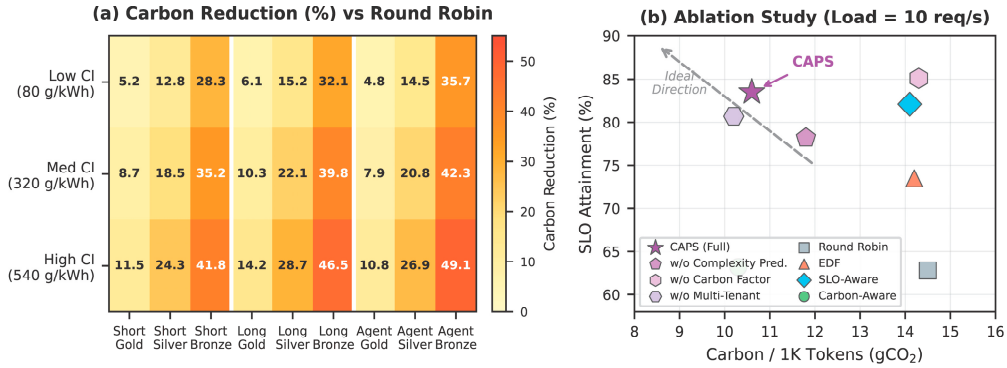


Figure 3. (a) Carbon reduction (%) of CAPS versus Round Robin across carbon intensity levels, request types, and SLO tiers. (b) Ablation study at 10 req/s showing the Pareto trade-off between SLO attainment and carbon efficiency; the arrow indicates the ideal optimization direction.

Table 1. Summary of key metrics at 10 req/s. Carbon/Goodput denotes carbon per 1K SLO-met tokens (^{gCO_2} /1K good tok); lower is better for all columns except SLO Att.

Method	SLO Att. (%) ↑	Carbon/1K (gCO_2) ↓	P95 Lat. (ms) ↓	Carbon/Goodput ↓
Round Robin	62.8	14.5	890	23.1
EDF	73.5	14.2	680	19.3

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

SLO-Aware	82.1	14.1	570	17.2
Carbon-Aware	63.2	10.3	1020	16.3
CAPS	**83.5**	**10.6**	**640**	**12.7**

CAPS achieves the lowest carbon cost per SLO-met token among all methods that maintain SLO attainment above 80%, as shown in the rightmost column of Table 1.

Impact of Carbon Intensity and Request Type

Figure 3(a) shows the carbon reduction achieved by CAPS over Round Robin across different combinations of carbon intensity level, request type, and SLO tier. The largest savings (49.1%) occur for Bronze-tier agentic pipelines during high-carbon periods, where CAPS aggressively defers execution to later low-carbon windows. Gold-tier short prompts show the smallest savings (5.2–11.5%), as these requests are routed to the low-latency pool regardless of carbon conditions.

Ablation Study

Figure 3(b) presents a scatter plot of SLO attainment versus carbon per 1K tokens for CAPS variants with individual components removed, all at 10 req/s. Removing the prompt complexity predictor ("w/o Complexity Pred.") reduces SLO attainment by 5.3 points (to 78.2%) because the scheduler can no longer anticipate SLO-violation risk. Removing the carbon factor ("w/o Carbon Factor") increases carbon by 34.9% (to 14.3 gCO_2) while slightly improving SLO attainment to 85.1%, confirming that the carbon objective introduces a controlled trade-off. Removing multi-tenant differentiation ("w/o Multi-Tenant") reduces SLO attainment by 2.8 points and marginally lowers carbon, since all requests are treated uniformly. The full CAPS achieves the best position in the SLO–carbon Pareto space.

Discussion and Limitations

Our simulation results suggest that integrating carbon awareness into LLM inference scheduling can yield meaningful emission reductions, particularly for workloads with heterogeneous SLO requirements. However, several limitations should be noted. First, our evaluation is simulation-based and does not capture all effects of a production deployment, such as GPU memory contention, KV-cache migration overhead across pools, and imperfect carbon forecasts. Second, the prompt complexity predictor is trained on ShareGPT-derived data and may underperform on out-of-distribution request patterns; its accuracy under distribution shift remains to be validated. Third, the three-pool resource split (8 GPUs each) is fixed; dynamic pool resizing could further improve efficiency but is left for future work. Fourth, we evaluate a single-region setting; geo-distributed deployments could exploit spatial carbon variability at the cost of cross-region latency. Finally, the reward function weights (α, β, λ) are set manually; adaptive tuning could improve robustness across diverse workload mixes.

Conclusions

We presented CAPS, a carbon-aware prompt scheduling framework for multi-tenant cloud LLM inference. By combining prompt complexity prediction, real-time carbon intensity monitoring, and SLO-differentiated pool routing, CAPS reduces carbon emissions per 1K tokens by 26.8% relative to round-robin scheduling in our trace-driven simulations, while maintaining SLO attainment comparable to a dedicated SLO-aware baseline. Ablation experiments confirm that each component contributes to the overall effectiveness. Translating these simulation-based gains to production systems will require addressing challenges such as carbon forecast uncertainty, dynamic pool sizing, and integration with existing continuous batching engines such as vLLM or Sarathi-Serve. These

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

results suggest that carbon awareness is a promising design dimension for future LLM serving schedulers, especially in multi-tenant deployments with heterogeneous SLO requirements.

References

1. C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang and C. Bai, "Sustainable AI: Environmental implications, challenges and opportunities," Proceedings of Machine Learning and Systems, vol. 4, pp. 795-813, 2022.
2. P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warrier, N. Mahalingam and R. Bianchini, "Characterizing power management opportunities for LLMs in the cloud," Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 207-222, 2024.
3. J. Dodge, T. Prewitt, R. Tachet des Combes, E. Odmark, R. Schwartz, E. Strubell, A. S. Luccioni, N. A. Smith, N. DeCario and W. Buchanan, "Measuring the carbon intensity of AI in cloud instances," Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency, pp. 1877-1894, 2022.
4. W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin and P. Shenoy, "CarbonScaler: Leveraging cloud workload elasticity for optimizing carbon-efficiency," Proceedings of the ACM on Measurement and Analysis of Computing Systems, vol. 7, no. 3, pp. 1-28, 2023.
5. K. Xu, D. Sun, H. Tian, J. Zhang and K. Chen, "GREEN: Carbon-efficient resource scheduling for machine learning clusters," 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25), pp. 999-1014, 2025.
6. M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz and O. Abboud, "GreenCourier: Carbon-aware scheduling for serverless functions," Proceedings of the 9th International Workshop on Serverless Computing, pp. 18-23, 2023.
7. B. Acun, B. Lee, F. Kazhamiaka, K. Maeng, U. Gupta, M. Chakkaravarthy, D. Brooks and C.-J. Wu, "Carbon Explorer: A holistic framework for designing carbon-aware datacenters," Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 118-132, 2023.
8. G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), pp. 521-538, 2022.
9. W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," Proceedings of the 29th Symposium on Operating Systems Principles, pp. 611-626, 2023.
10. A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pp. 117-134, 2024.
11. P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA), pp. 118-132, 2024.
12. L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica and J. E. Gonzalez, "SGLang: Efficient execution of structured language model programs," Advances in Neural Information Processing Systems, vol. 37, pp. 62557-62583, 2024.
13. D. Maji, P. Shenoy and R. K. Sitaraman, "CarbonCast: Multi-day forecasting of grid carbon intensity," *Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 198–207, 2022.
14. X. Yang, S. Li, K. Wu, Z. Wang, Y. Tang and Y. Li, "Adaptive anomaly detection in microservice systems via meta-learning," 2026.
15. N. Chen, Y. Zhang, W. Wang, Z. Pan, Y. Wang and Y. Lu, "CoReAD: Context-aware retrieval-augmented deep anomaly detection for evolving business tabular data," 2026.
16. H. Jiang, F. Qin, J. Cao, Y. Peng and Y. Shao, "Recurrent neural network from adder's perspective: Carry-lookahead RNN," *Neural Networks*, vol. 144, pp. 297–306, 2021.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 April 2026 doi:10.20944/preprints202604.1399.v1

17. X. Zhang, Q. Wang and X. Wang, "Joint cross-modal representation learning of ECG waveforms and clinical reports for diagnostic classification," *Transactions on Computational and Scientific Methods*, vol. 6, no. 2, 2026.
18. N. Chen, Y. Zhang, W. Wang, Z. Pan, Y. Wang and Y. Lu, "CoReAD: Context-aware retrieval-augmented deep anomaly detection for evolving business tabular data," 2026.
19. J. Huang, J. Zhan, Q. Wang, J. Jia and B. Zhang, "Stable fault diagnosis under data imbalance via self-supervised learning in industrial IoT," 2026.
20. C. Wen, A. Zhu, R. Long, H. Huang, J. Jiang and C. S. Lee, "CalibJudge: Calibrated LLM-as-a-judge for multilingual RAG with uncertainty-aware scoring," 2026.
21. C. Zhang, H. Zhu, A. Zhu, J. Liao, Y. Xiao and Z. Zhang, "Deep learning approach for protocol anomaly detection using status code sequences," 2026.
22. X. Yang, S. Sun, Y. Li, Y. Xing, M. Wang and Y. Wang, "CaliCausalRank: Calibrated multi-objective ad ranking with robust counterfactual utility optimization," *arXiv preprint arXiv:2602.18786*, 2026.
23. N. Lyu, Y. Wang, Z. Cheng, Q. Zhang and F. Chen, "Multi-objective adaptive rate limiting in microservices using deep reinforcement learning," *Proceedings of the 4th International Conference on Artificial Intelligence and Intelligent Information Processing*, pp. 862–869, 2025.
24. Z. Huang, J. Yang, S. Li, C. Zhang, J. Chen and C. Xu, "Shared representation learning for high-dimensional multi-task forecasting under resource contention in cloud-native backends," *arXiv preprint arXiv:2512.21102*, 2025.
25. J. Jiang, C. Shao, C. Zhang, N. Lyu and Y. Ni, "Adaptive AI spatiotemporal modeling with dependency drift awareness for anomaly detection in large-scale clusters," 2025.
26. Y. Wang, R. Yan, Y. Xiao, J. Li, Z. Zhang and F. Wang, "Memory-driven agent planning for long-horizon tasks via hierarchical encoding and dynamic retrieval," 2025.
27. H. Zhuang, N. Lyu, R. Wei, W. Huang, J. Kou and W. Huang, "TokenPool-Scheduler: Token-level GPU pooling and resource slicing for multi-model co-location," 2026.
28. K. Zeng, Z. Huang, Y. Yang, R. Meng, S. Y. Huang and X. Zhang, "TokenFlow: Token-level GPU sharing and adaptive scheduling for multi-model concurrent LLM inference," Environments, vol. 21, p. 24.
29. A. Zhu, W. Liu, Z. Li, C. Wen, J. Qiu and Z. Liu, "ArcheScale-Guard: Archetype-aware predictive autoscaling with uncertainty quantification for serverless computing," 2026.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.